DataSource in Terraform

To Begin with the Lab

Summary of the Lab

In this lab, we learn how to use Terraform Data Sources to fetch information dynamically from AWS instead of hardcoding values in Terraform files. Rather than manually entering details such as Availability Zones or AMI IDs, Terraform can automatically retrieve the latest information from AWS at runtime.

The lab demonstrates how to:

Create and use Terraform data sources.

Dynamically fetch AWS Availability Zones.

Select a specific Availability Zone from the available list.

Retrieve the latest Ubuntu AMI automatically.

Deploy infrastructure using dynamically fetched values instead of fixed values.

Understand how AWS providers, AMIs, and filters work together.

In simple terms, this lab shows how Terraform can "ask AWS for information" before creating resources, making infrastructure code more flexible, reusable, and easier to maintain.

- Create a new folder named Demo Data Source.
- createinstance.tf
- provider.tf
- variables.tf
- Initialize Terraform
- Command:
- terraform init
- Verify Current Plan
- Command:
- terraform plan
- observe that the Availability Zone is not automatically selected.
- Run Terraform Plan Again
- Command:
- terraform plan
- Verify that Terraform now displays a valid Availability Zone.
- Test Different Availability Zones
- Change the index value and run the plan again.
- Example:
- availability_zone = data.aws_availability_zones.available.names[0]
- Command:

- terraform plan
- Understand Invalid Index Errors
- If an unavailable index is used:
- availability_zone = data.aws_availability_zones.available.names[7]
- Terraform returns an error because that Availability Zone does not exist.
- Add an AMI Data Source
- Create a data source to retrieve the latest Ubuntu AMI.
- Code:

```
data "aws_ami" "ubuntu" {
  most_recent = true

  owners = [
    "099720109477"
  ]

  filter {
    name = "name"

    values = [
      "ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server-*"
    ]
  }

  filter {
    name = "virtualization-type"

    values = [
      "hvm"
    ]
  }
}
```

- Use the Dynamic AMI
- Replace the hardcoded AMI value.
- Code:

```
ami = data.aws_ami.ubuntu.id
```

- Validate the Configuration
- Run Terraform Plan to verify that the latest Ubuntu AMI is selected automatically.
- Command:

```
terraform plan
```

- Test with Different AWS Regions
- Provide a region at runtime and let Terraform fetch the correct AMI automatically.
- Command:

```
terraform plan -var="region=us-east-1"
```

- Observe Dynamic Results
- Terraform automatically retrieves:
- Latest Ubuntu AMI
- Correct Availability Zone
- without requiring hardcoded values.